

TSLOMS 信号灯检测器接入操作手册

适用范围: TSLOMS (Traffic Signal Light Operation and Maintenance System)
本文档说明信号灯检测器 (真实硬件 / CSV 回放 / Mock 模拟) 如何接入本系统, 覆盖系统登录、接入查看、硬件侧要求与操作、MQTT 环境配置、二进制协议帧构造、Topic 约定、故障码定义与完整验证流程。

本实例为腾讯云生产环境: <http://129.211.223.113:8092/> 已对外运行 (env=production), 本文档按该生产实例编写。

0. 完整操作流程 (先看这张图)

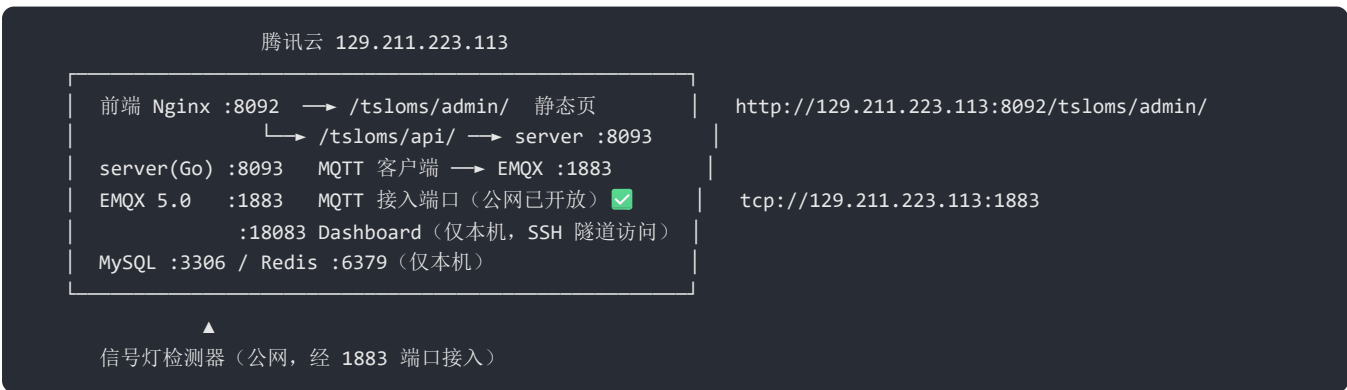
① 登录系统 → ② 进入「检测器接入」查看状态 → ③ 硬件侧配置/操作 → ④ 系统侧配置 → ⑤ 验证接入结果

步骤	操作	完成标准	参考章节
①	浏览器打开 http://129.211.223.113:8092/tsloms/admin/ , 用账号登录	进入仪表盘	第 1 节
②	侧边栏「检测器接入」查看 Broker 连接、订阅 Topic、在线设备数	页面可见 Broker「已连接」	第 2 节
③	硬件侧配置公网 MQTT 地址与账号, 编码协议帧上报	设备能连上 EMQX	第 3、4 节
④	系统侧确认 <code>MQTT_*</code> 配置与 EMQX 认证 (生产已就绪)	Broker「已连接」	第 5 节
⑤	用「设备管理」「故障管理」「工单管理」核对	台账/故障/工单齐全	第 10 节

调试期建议: 先走「Mock 模拟」或「CSV 导入」验证链路 (第 8、9 节), 链路通畅后再接真实硬件, 可大幅缩短排障时间。

0.1 生产环境 (腾讯云) 接入总览

本实例部署拓扑 (已实测验证):



项	状态 (2026-08-18 实测)
前端/API http://129.211.223.113:8092/	✅ HTTP 200, <code>env=production</code>
MQTT 公网端口 129.211.223.113:1883	✅ TCP 开放, 检测器可直接连接
EMQX Dashboard 18083	公网不通 (安全收敛), 需 SSH 隧道到本机访问

项	状态 (2026-08-18 实测)
服务端 MQTT 账号	<code>MQTT_USERNAME</code> (默认 <code>tsloms</code>) + <code>MQTT_PASSWORD</code> 连接本机 EMQX

关键结论：**检测器直接连 `tcp://129.211.223.113:1883` 即可接入**，前提是拿到 EMQX 上有效的 MQTT 用户名/密码（见第 3 节）。

1. 系统登录

1.1 访问地址

- 生产前端管理台：`http://129.211.223.113:8092/tsloms/admin/`
- 入口根路径 `http://129.211.223.113:8092/` 自动 302 跳转到 `.../admin/`
- 后端 API 基址：`http://129.211.223.113:8092/tsloms/api/v1/` (Nginx 反代到本机 server :8093)

1.2 账号与密码

账号	角色	说明
<code>admin</code>	系统管理员 (admin)	首次部署由 <code>ADMIN_INIT_PASSWORD</code> 指定；未指定则系统生成随机强密码并在 服务端启动日志打印一次
<code>419116</code>	超级管理员 (super_admin)	由 <code>SUPER_ADMIN_PASSWORD</code> 指定或启动日志打印随机密码；含全部权限与模块设置

密码为空时务必从服务端启动日志中记录初始随机密码，登录后立即修改。
若已存在 admin 账号（重复启动），SeedAdmin 幂等跳过，不会覆盖密码。

1.3 登录步骤

- 打开前端地址，跳转 `/login` 登录页。
- 输入账号（用户名或手机号）、密码。
- 填写**算术验证码**：页面显示算式（如 `2+8=?`），输入答案（`10`），点击算式可刷新。
- 点击「登录」，成功后进入仪表盘。
- 无账号时可点「还没有账号？立即注册」走 `/register` 自助注册（按角色审核）。

2. 查看「检测器接入」

2.1 入口

- 侧边栏主菜单 → 「**检测器接入**」（核心基础模块，恒启展示，无需购买可选模块）
- 路由：`/access`；前端菜单标题「检测器接入（真实硬件/CSV/Mock）」

2.2 页面内容

进入后默认展示四张概览卡片：

卡片	字段	含义
真实硬件接入 (MQTT)	已连接 / 未连接	服务端与 EMQX Broker 的连接状态
	在线设备 / 活跃检测器	在线设备总数 / 近 30 分钟有上行的设备数
CSV 数据导入	可用	批量回放工具是否可用 (恒可用)
Mock 数据模拟	可用	本地模拟工具是否可用 (恒可用)
订阅 Topic	trafficLight/{网络号}/{站点号}/{硬件ID}/U	服务端实际订阅的主题模式

- 下方三个 Tab:
- **真实硬件接入**: 接入方式说明、当前连接状态、订阅 Topic、在线设备数、接入步骤
 - **CSV 数据导入**: 粘贴 CSV 批量回放 (第 8 节)
 - **Mock 数据模拟**: 单条协议帧模拟投递 + 故障码速查表 (第 9 节)

3. 硬件侧要求 (接入前置条件)

3.1 网络要求 (生产实例实测)

项	要求
传输层	支持 MQTT over TCP, MQTT 3.1.1
目标地址	tcp://129.211.223.113:1883 (腾讯云公网, 已实测开放)
鉴权	必须配置 EMQX 上有效的 MQTT 用户名/密码 (生产已启用密码认证, 见 3.4)
网络/站点	设备具备网络号、站点号, 用于拼装 Topic

端口 1883 若被腾讯云安全组/本机防火墙拦截, 需在控制台放行:
安全组 → 入站规则 → 添加 TCP 1883 (建议按需限制来源 IP, 见第 11 节运维)。

3.2 硬件能力要求

项	要求
协议帧	按第 6 节 CMD_FRAME 二进制帧编码 (16 字节头 + 事件包)
事件上报	能上报 CHECKIN (签到) / ALARM (告警) / POWER_ON (上电) 三类事件
故障检测	能产出 errCode (-1~-14, 见 6.5) 与 ledState、三相电流
硬件 ID	具备出厂唯一 ledHwId (Topic 段为十六进制大写 ASCII, EVENT_RECORD 内为 uint32)
校时	能解析时间同步回应 (userVal = UTC+8 epoch 秒) 并校准本地时间

3.3 建议的初始化流程 (硬件侧)

- 出厂烧录: 写入唯一 LedHwID、固件版本 swVer、配置版本 confVer。
- 上电: 先发一帧 POWER_ON (0x03) 报告启动, 后台建账并回时间同步。
- 正常工作: 按周期 (如 60s) 发 CHECKIN (0x00) 心跳签到。
- 异常: 检测到灯组故障立即发 ALARM (0x01) 携带 errCode/灯态/电流。

3.4 生产 EMQX 账号 (检测器凭据)

生产 EMQX 已启用**密码认证** (built_in_database) 。任何 MQTT 连接 (含服务端与检测器) 都需通过认证:

项	值
认证方式	MQTT 用户名/密码 (EMQX 5.0 内置数据库)
服务端账号	<code>MQTT_USERNAME</code> (默认 <code>tsloms</code>) + <code>MQTT_PASSWORD</code> (见 5.1)
检测器账号	需在 EMQX 中 单独创建 , 与生产配置解耦

说明: 生产 `docker-compose.prod.yml` 只固化了一个 MQTT 认证用户 (服务端用)。
检测器应使用独立账号, 避免与服务端共用凭据、便于按设备审计与吊销。

创建检测器账号 (二选一):

- **Dashboard 图形化** (推荐): SSH 登录服务器后执行
`ssh -L 18083:127.0.0.1:18083 root@129.211.223.113`, 浏览器开
`http://127.0.0.1:18083` → 访问控制 → 认证 → 内置数据库 → 添加用户。
- **命令行 (EMQX API)**: 在服务器本机执行
`bash curl -X POST 'http://127.0.0.1:18083/api/v5/authentication/1/users' \ -u admin:'<EMQX_DASHBOARD_PASSWORD>' \ -H 'Content-Type: application/json' \ -d '{"user_id":"det123","password":"<强密码>"}'`

创建后把 `det123 / <密码>` 配置到检测器 (见 4.1)。

⚠ **账号长度约束 (PDF 4.1/4.2)**: 设备配置文件中 `mqttUserName / mqttPassword` **最长 8 位**。
创建 EMQX 检测器账号时建议控制在 8 位以内 (超出部分设备配置工具无法写入)。

3.5 检测器参数配置 (trafficLightConf 工具)

检测器连接 MQTT 所需的**全部参数写进设备配置**, 由厂商提供的 linux 工具 `trafficLightConf` 下发 (PDF 第 4 章, 依赖 `libpaho-mqtt3a` 运行库)。

三个文件:

文件	作用
<code>trafficLightConf</code>	主配置程序 (linux-x86-64)
<code>mqtt.ini</code>	工具自身连服务器信息
<code>trafficLight.ini</code>	写入设备的配置模板

`mqtt.ini` (登录服务器信息) 示例 (PDF 4.1):

```
[mqtt]
mqttServerIp=129.211.223.113 ;MQTT 服务器 IP
mqttServerPort=1883 ;MQTT 服务器端口
mqttUserName=det123 ;登录用户名 (最长 8 位)
mqttPassword=xxxxxxx ;登录密码 (最长 8 位)
mqttTopicPrefix=trafficLight ;topic 前缀 (一般不修改)
networkCode=0 ;网络号 (默认 0, 不修改)
stationCode=0 ;站点号 (默认 0, 不修改)
```

`trafficLight.ini` (写入设备的参数模板) 关键项 (PDF 4.2):

参数	示例	含义
<code>confVer</code>	<code>26081801</code>	配置版本号 (16 进制 YYMMDDBB)
<code>mqttServerUserIp</code>	<code>129.211.223.113</code>	设备连接的 MQTT 服务器 IP
<code>mqttServerPort</code>	<code>1883</code>	MQTT 端口

参数	示例	含义
<code>mqttUserName</code> / <code>mqttPassword</code>	<code>det123</code>	设备登录账号（最长 8 位）
<code>networkCode</code> / <code>stationCode</code>	<code>0</code>	网络号 / 站点号
<code>mqttTopicPrefix</code>	<code>trafficLight</code>	topic 前缀
<code>checkinMin</code>	<code>2</code>	签到周期（分钟）
<code>ledMaxPeriodSecR/Y/G</code>	<code>100/5/120</code>	红/黄/绿最长周期秒数（超时上报）
<code>powerLossSec</code>	<code>200</code>	三灯全灭超过该秒数上报断电

下发命令 (PDF 4.3) :

```
./trafficLightConf <hwId> [options]
# 用模板配置 hwId=1114006C 的设备
./trafficLightConf 1114006C
# 临时改参数配置（不写回 ini 文件）
./trafficLightConf 1114006C --mqttServerUserIp 129.211.223.113 --mqttServerPort 1883 --mqttUserName det123
```

流程：工具订阅 `trafficLight/0/0/+/U`，向 `trafficLight/0/0/<hwId>/D` 发送 `cmd=0x20` (CMD_UPDATE_CONFIG) 配置帧。**成功标志**：设备收到新配置后自动重启，上报的 `confVer` 与发出的一致（见 `/access` 页面设备台账或报文日志）。

4. 设备侧接入（真实硬件）

4.1 设备需要做的配置

- 1. 连接 Broker：
 - Broker 地址：`tcp://129.211.223.113:1883`（腾讯云公网）
 - 用户名/密码：使用第 3.4 节在 EMQX 中创建的**检测器专用账号**（如 `det123`）
- 2. 发布 Topic：`trafficLight/{网络号}/{站点号}/{硬件ID}/U`
 - `{网络号}` `{站点号}` `{硬件ID}` 均为**十六进制大写 ASCII 码**（PDF 前言，例如 `trafficLight/0/0/11130000/U`、`trafficLight/0/0/1114006C/D`）；网络号 0~254、站点号 0~65534，当前默认均为 0，不建议修改
 - `{硬件ID}`：设备出厂唯一硬件编号（如 `1114006C`），后台据 EVENT_RECORD 内 `ledHwId` (uint32) 转大写十六进制 uuid 入库（如 `0x00001F41` → `00001F41`）
- 3. payload 为二进制协议帧（见第 6 节），定时发送：
 - 正常时周期性发 **CHECKIN**（签到）
 - 检测到信号灯异常时发 **ALARM**（告警）
 - 上电/重启完成后发 **POWER_ON**（上电报告）

4.2 设备无需额外动作的事

- 新硬件 ID 首次上报自动创建设备台账（自动建账）
- 时间同步回应、固件查询回应由后台自动下发到 `/D` Topic
- 故障研判、故障记录、维修工单由系统自动完成

5. 系统侧 MQTT 配置

5.1 配置项（环境变量）

服务端通过环境变量读取 MQTT 配置，默认值见 `internal/config/config.go`，部署模板见 `deploy/.env.example`：

环境变量	生产本实例实际值	默认值	说明
MQTT_BROKER	tcp://127.0.0.1:1883（本机 EMQX）	tcp://localhost:1883	Broker 地址（EMQX 默认 1883）
MQTT_USERNAME	tsloms	tsloms	服务端连接 EMQX 的用户名
MQTT_PASSWORD	生产设置的强密码	空	服务端连接 EMQX 的密码
MQTT_CLIENT_ID	tsloms-server	tsloms-server	服务端客户端标识
MQTT_TOPIC_PREFIX	trafficLight	trafficLight	Topic 前缀，可自定义

部署方式（生产）：

- server 以 systemd 运行（`deploy/systemd/tsloms-server.service`），凭据统一放 `/etc/tsloms/tsloms.env`（0600），其中含 `MQTT_BROKER=tcp://127.0.0.1:1883`、`MQTT_USERNAME=tsloms`、`MQTT_PASSWORD=<强密码>`。

- MySQL / Redis / EMQX 以 `docker-compose.prod.yml` 部署，全部端口仅绑定 `127.0.0.1`（EMQX 1883 映射到本机，公网经腾讯云端口转发/安全组放行）。

- 前端静态页与 API 由 Nginx :8092 对外（`deploy/nginx/tsloms.conf`）。

修改任一 `MQTT_*` 后需重启：`sudo systemctl restart tsloms-server`。

5.2 订阅 Topic

服务端启动后自动订阅（`cmd/server/main.go` L84）：

```
{MQTT_TOPIC_PREFIX}/{+}/{+}/{+}/U      默认即 trafficLight/{+}/{+}/{+}/U
```

- QoS = 1
- 每段含义：`trafficLight/{网络号}/{站点号}/{硬件ID}/U`
- 上行一律以 `/U` 结尾；后台对同一帧的下行回应发往将 `/U` 替换为 `/D` 的 Topic。

5.3 接入验证入口

前端「检测器接入」页面（路由 `/access`）提供：

- 真实硬件接入状态总览（Broker 连接、订阅 Topic、在线设备数、近 30 分钟活跃检测器数）
- CSV 导入 / Mock 模拟两个调试工具

生产访问：登录后打开 `http://129.211.223.113:8092/tsloms/admin/#/access`。

若该页「真实硬件接入」显示「已连接」，说明服务端已连上本机 EMQX；检测器接入后再看在线设备数增长。

6. 二进制协议帧定义

6.1 CMD_FRAME 命令帧（16 字节固定头 + 变长数据）

偏移	长度	字段	说明
0	1	token	魔术字，固定 <code>0x55</code>
1	1	cmd	命令类型（见 6.2）

偏移	长度	字段	说明
2	1	ver	协议版本, 当前 0x10
3	1	checksum	校验和: 整包所有字节按 uint8 累加, 结果 (自然溢出) 必须等于 0xFF
4	4	swVer	软件版本号 (大端位域), 见下方位域说明
8	2	cmdSeq	包序号, 每发一帧 +1 (大端)
10	2	datLen	数据部分长度 (大端)
12	4	userVal	用户自定义 (时间同步回应使用, 大端)
16	datLen	data	变长数据, 按 cmd 决定结构

swVer 位域编码 (PDF 表 3-1) :

```
bit[31:28] = 主版本 major
bit[27:24] = 次版本 minor
bit[23:18] = 编译年份 (2000+n)
bit[17:14] = 编译月份
bit[13:8]  = 编译日
bit[7:0]   = 当天编译序号 (新的一天从 1 开始)
```

6.2 命令类型 (cmd)

常量	值	方向	含义
CmdCheckin	0x00	设备→服务器	定时签到
CmdAlarm	0x01	设备→服务器	故障报警
CmdPowerOn	0x03	设备→服务器	上电/重启完成报告
CmdUpdateConfig	0x20	服务器→设备	下发配置更新 (trafficLightConf 用)
CmdCheckFW	0x30	设备→服务器	查询是否有新固件
CmdGetFW	0x31	设备→服务器	请求固件升级数据
CmdReboot	0x7F	服务器→设备	重启指令
CmdAckFlag	0x80	—	回应帧标志 (bit7=1 回应帧, cmd 或上该位)

帧命令定义范围 0x00-0x7F; 最高位=0 表示请求帧, 最高位=1 表示回应帧。
系统当前对上行处理 CHECKIN / ALARM / POWER_ON / CHECK_FW / GET_FW。

6.3 事件包 EVENT_PAK (CHECKIN / ALARM 的 data 部分)

偏移	长度	字段	说明
0	2	eventRecordNum	事件记录数量 (大端)
2	2	datLen	事件记录总长 = 记录数 × 24 (大端)
4	N×24	records	EVENT_RECORD 数组

6.4 事件记录 EVENT_RECORD (24 字节, 1 字节对齐)

偏移	长度	字段	说明
0	4	ledHwId	设备硬件 ID（出厂唯一，大端）
4	4	subHwId	子灯组 ID（同一设备不同灯组，当前版本未启用，大端）
8	4	swVer	软件版本号（大端，位域见 6.1）
12	4	confVer	配置版本号 <code>0xYYMMDDBB</code> （大端），例 <code>0x26081801</code> = 26年08月18日01版
16	1	ledState	灯组状态：0=红 / 1=黄 / 2=绿 / -1=未知
17	1	errCode	错误码（见 6.5，int8）
18	2	currentR	红灯电流值 0-2048（大端）
20	2	currentY	黄灯电流值 0-2048（大端）
22	2	currentG	绿灯电流值 0-2048（大端）

注：PDF 结构体定义中 `EVENT_RECORD` 无 `reserved` 字段（共 24 字节），但 PDF 3.2 CHECKIN 举例图在 `confVer` 与 `errCode` 之间画了一个 `reserved` 字节（25 字节），**协议文本自相矛盾**。本系统按 24 字节结构体定义实现。

6.5 故障错误码（errCode）

错误码	含义	故障类型	等级
0	工作正常	—	—
-1	红灯周期全灭	lamp_off（灭灯）	critical
-2	黄灯周期全灭	lamp_off	critical
-3	绿灯周期全灭	lamp_off	critical
-4	红黄同亮	abnormal_on（异常同亮）	critical
-5	红绿同亮	abnormal_on	critical
-6	黄绿同亮	abnormal_on	critical
-7	红黄绿同亮	abnormal_on	critical
-8	红灯亮灯超时	timeout（超时）	normal
-9	黄灯亮灯超时	timeout	normal
-10	绿灯亮灯超时	timeout	normal
-11	红灯缺亮（暂未实现）	dim（缺亮）	normal
-12	黄灯缺亮（暂未实现）	dim	normal
-13	绿灯缺亮（暂未实现）	dim	normal
-14	断电（超过设定阈值）	power_loss（断电）	critical

常量定义见 `internal/faultcode/faultcode.go`；协议解析见 `internal/mqtt/parser.go`。

6.6 下行回应（服务器 → 设备，发往 `/D` Topic）

触发	回应	说明
CHECKIN (0x00) / POWER_ON (0x03)	<code>cmd\ 0x80</code> 时间 同步回应	<code>userVal</code> = 当前 epoch 秒 (UTC+8 修正) , PDF 3.2/3.4
CHECK_FW (0x30)	<code>0x30\ 0x80</code> 固件 查询回应	PDF 3.1.4: data 为 FIRMWARE_INFO_DAT (swVer 4B + fwLen 2B + fwChecksum 1B + reserved 1B) 。 本系统简化: data 仅目标固件位域值 4 字节, 0 表示无新版本
GET_FW (0x31)	<code>0x31\ 0x80</code> 固件 数据回应	PDF 3.1.5: data 为 FIRMWARE_PAK (target 1B + datLen 1B + offset 2B + dat[]) 。 本系统简化: 当前与固件查询回应一致 (目标位域值 4 字节)
ALARM (0x01)	无需回应	PDF 3.3: 服务器收到报警无需回应

固件完整升级包下发 (FIRMWARE_PAK 逐包推送) 为协议定义能力, 当前生产版本未启用;
若硬件需完整升级流程, 需与固件模块联调。

7. 消息处理链路



8. CSV 批量导入 (无硬件回放)

在 `/access` 页面「CSV 数据导入」粘贴如下格式 (首行表头可选) :

```
hw_id,cmd,err_code,led_state,current_r,current_y,current_g
9001,alarm,-5,0,600,50,40
9002,checkin,-1,0,0,0,0
9003,alarm,-14,0,0,0,0
```

- 每行构造一条合法协议帧并回放, 走与真实硬件相同的研判链路
- `cmd` : checkin / alarm / power_on
- `err_code` : 见 6.5 故障码表
- 后端接口: `POST /api/v1/access/csv/import` (`handler/access.go`)

9. Mock 数据模拟（无需 Broker）

在 `/access` 页面「Mock 数据模拟」填写 硬件ID / 命令 / 错误码 / 灯态 / 三相电流，点「发送」即投递一条合法协议帧：

- 后端接口：`POST /api/v1/access/mock/send`
- 实现：`mqtt.DispatchFrame`（`internal/mqtt/simulate.go:135`）构造 paho 消息直接走 `HandleMessage`，无硬件、无 Broker 在线也可用，适合开发调试与演示。

10. 快速验证清单

1. 浏览器打开 `http://129.211.223.113:8092/tsloms/admin/`，用 admin / 419116 账号登录
2. 侧边栏「检测器接入」→「真实硬件接入」显示 Broker「已连接」（服务端已连本机 EMQX）
3. 检测器用第 3.4 节账号连 `tcp://129.211.223.113:1883` 并发布一帧 CHECKIN 后，「设备管理」出现该设备台账且在线
4. 检测器发布一帧 ALARM（errCode 非 0）后：
 - 「故障管理」出现对应故障记录（类型/等级与 6.5 表一致）
 - 「工单管理」出现自动生成的维修工单
5. 时间同步回应已自动下发（SSH 隧道看 EMQX Dashboard 的 `/D` Topic 流量）

11. 生产运维（腾讯云专项）

11.1 连接排障速查

现象	排查
检测器连不上 1883	① 腾讯云安全组是否放行 TCP 1883；② 本机 <code>firewall-cmd --list-ports / ufw</code> ；③ <code>nc -vz 129.211.223.113 1883</code> 从外网验证
能连但认证失败	EMQX 认证用户/密码是否正确（第 3.4 节）；Dashboard 内置数据库核对
服务端「未连接」	<code>sudo systemctl status tsloms-server</code> ； <code>/etc/tsloms/tsloms.env</code> 的 <code>MQTT_BROKER/USERNAME/PASSWORD</code> 是否正确
收到帧但无设备台账	检查 Topic 是否为 <code>trafficLight/{网络}/{站点}/{硬件ID}/U</code> （五段， <code>/U</code> 结尾）；帧 checksum 是否 0xFF（见 6.1）

11.2 安全基线（生产已启用）

- EMQX 1883 公网开放仅用于检测器接入；Dashboard 18083 只绑定 127.0.0.1，公网不通，必须 SSH 隧道访问
- 建议安全组将 1883 的入站来源限制为实际部署检测器的网段（或按需开放）
- 检测器账号独立于服务端账号（避免共用 `tsloms` 凭据），便于吊销与审计
- `MQTT_PASSWORD`、`EMQX_DASHBOARD_PASSWORD` 只存于 `/etc/tsloms/tsloms.env`（0600），不落仓库